

1. Présentation Générale du Projet

1.1 Objectif

Développer une plateforme web dynamique permettant au COUS-AC de dématérialiser la gestion de ses services (hébergement en cabines, restauration universitaire, et administration) sur ses différents sièges universitaires (UAC, INSTI-Lokossa, etc.).

1.2 Acteurs du Système (Rôles & Permissions)

C'est ici que la cohérence de la structure est indispensable. Chaque utilisateur aura un rôle précis stocké en base de données :

- **Étudiant** : Client principal. Il consulte les cabines, fait sa demande, achète ses tickets de restaurant en ligne (via Mobile Money) et consulte ses reçus.
 - **Intendant (Chef de Siège)** : Super-administrateur du siège d'une université donnée. Il valide les attributions de cabines et supervise le personnel.
 - **Comptable** : Gère l'aspect financier (validation des paiements de dortoirs, encaissement des ventes de tickets de restaurant, bilans financiers).
 - **Secrétaire** : Enregistre les dossiers des étudiants, gère le courrier administratif et met à jour les annonces officielles.
 - **Chef de Cabine (CC)** : Étudiant responsable d'un dortoir spécifique. Il signale les pannes ou incidents à l'intendant.
 - **Chef de la Sécurité** : Gère la liste des agents de sécurité, planifie les rondes et reçoit les rapports d'incidents (vols, intrusions).
 - **Chef des Agents d'Entretien** : Planifie le nettoyage des espaces communs et attribue les tâches aux agents.
 - **Cuisiniers & Agents du Restaurant** : Mettent à jour le menu du jour et scannent/valident les tickets des étudiants lors du service.
-

2. Spécifications Fonctionnelles (Les Modules du Site)

Pour que ton site PHP soit structuré, nous allons le diviser en **4 grands modules** :

Module A : Authentification et Profils (Tous les acteurs)

- Inscription des étudiants avec vérification du matricule.
- Connexion sécurisée par session PHP (`$_SESSION`).
- Gestion des rôles (Si `role === 'etudiant'`, redirection vers l'espace étudiant ; si `role === 'comptable'`, redirection vers le dashboard financier).

Module B : Le "E-Commerce" du Restaurant Universitaire

- **Côté Étudiant :** * Visualisation du menu du jour (généré en SQL/PHP).
 - Achat de tickets de restaurant (Matin, Midi, Soir).
 - Génération d'un **QR Code unique** par ticket (généré en JavaScript ou via une bibliothèque PHP) servant de preuve d'achat.
- **Côté Agents du Restaurant :**
 - Interface simple pour scanner/valider le ticket de l'étudiant (changement de statut du ticket en base de données : `utilise = 1`).

Module C : Gestion de l'Hébergement (Les Cabines)

- **Côté Étudiant :** Formulaire de demande de dortoir (choix du campus/siège, téléversement des pièces justificatives).
- **Côté Secrétaire/Intendant :** Liste des demandes, bouton pour "Accepter" ou "Refuser".
- **Côté CC (Chef de Cabine) :** Interface pour signaler un problème technique (ex: ampoule grillée, plomberie gâtée).

Module D : Gestion du Personnel et Logistique (Nouveaux éléments cruciaux pour la cohérence)

- **Dashboard Sécurité :** Registre numérique des incidents sur le campus.
 - **Dashboard Entretien :** Planning de propreté des bâtiments.
-

3. Architecture Technique et Base de Données (SQL)

Puisque tu codes en PHP natif, ta base de données doit être parfaitement normalisée pour éviter les bugs. Voici le schéma relationnel logique que tu devras créer :

Tables SQL principales à créer :

1. **universites** (id, nom_universite, ville) -> Ex: (1, 'UAC', 'Calavi'), (2, 'INSTI', 'Lokossa')
2. **utilisateurs** (id, matricule_ou_identifiant, nom, prenom, email, mot_de_passe, role, universite_id)
3. **cabines** (id, numero_cabine, batiment, capacite, universite_id, intendant_id, cc_id)
4. **demandes_hebergement** (id, etudiant_id, cabine_id, statut [en_attente, valide, rejete], date_demande)
5. **menus** (id, jour, repas [matin, midi, soir], description_repas, prix)

6. **tickets_restaurant** (id, etudiant_id, menu_id, code_unique_qr, statut_utilisation [0 ou 1], date_achat)
 7. **incidents** (id, signale_par_id, description, type [technique, securite], statut [regle, en_cours])
-

4. Contraintes de Sécurité & Performances (Spécial PHP Natif)

Coder sans framework demande de la discipline. Tu dois impérativement respecter ces règles dans ton code backend :

- **Injections SQL** : Interdiction d'utiliser `$db->query("SELECT * FROM... WHERE id = " . $id)`. Tu utiliseras **exclusivement** les requêtes préparées avec PDO (`$stmt->execute()`).
 - **Hachage des mots de passe** : Utilisation de la fonction native `password_hash($password, PASSWORD_BCRYPT)` à l'inscription et `password_verify()` à la connexion.
 - **Protection des pages** : Création d'un fichier de vérification (ex: `check_session.php`) inclus en haut de chaque page pour empêcher un étudiant d'accéder au tableau de bord du comptable en changeant simplement l'URL.
-

5. Spécifications Visuelles (Tailwind CSS & JS)

- **Design** : Interface épurée inspirée des chartes institutionnelles (Vert/Jaune/Rouge subtil pour le Bénin, ou Bleu/Blanc professionnel).
- **Responsive** : Le site doit être **Mobile-First** avec Tailwind, car 95% des étudiants utiliseront leur smartphone pour acheter leurs tickets ou vérifier leur attribution de cabine.
- **JavaScript (AJAX)** : Lorsque l'étudiant clique sur "Acheter un ticket", le panier doit s'incrémenter en temps réel sans recharger toute la page pour offrir une expérience fluide.